

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
SOLID PRINCIPLES



Disusun Oleh:

Arif Mufti Tharsa (105224041)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pada THT Modul 12 ini, tugas yang dikerjakan adalah menganalisis masalah arsitektur pada sistem KRS SIAKAD. Di soal, sistem lama punya class besar bernama SistemKRSManager yang mengurus terlalu banyak hal, mulai dari validasi prasyarat, hitung UKT, cetak draft PDF, sampai simpan data ke MySQL. Masalah seperti ini bikin kode susah dirawat, karena perubahan kecil di satu bagian bisa merusak bagian lain.

Laporan ini berisi analisis pelanggaran SOLID dan rancangan ulang kode dengan memecah tanggung jawab ke beberapa file Java. Kode yang ditulis bukan sistem SIAKAD asli dan tidak benar-benar terhubung ke database atau membuat PDF sungguhan. Bagian database dan PDF dibuat sebagai simulasi agar konsep arsitekturnya terlihat jelas.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kode	String	Menyimpan kode mata kuliah.
2	nama	String	Menyimpan nama mata kuliah atau mahasiswa.
3	sks	int	Menyimpan jumlah SKS mata kuliah.
4	nim	String	Menyimpan NIM mahasiswa.
5	skemaUKT	SkemaUKT	Menyimpan aturan UKT yang dipakai mahasiswa.
6	idKRS	String	Menyimpan ID KRS.
7	mahasiswa	Mahasiswa	Menyimpan mahasiswa pemilik KRS.
8	daftarMataKuliah	List<MataKuliah>	Menyimpan daftar mata kuliah dalam KRS.
9	BATAS_SKS	int	Batas maksimal SKS pada validator sederhana.
10	repository	KRSRepository	Menyimpan dependency penyimpanan KRS.
11	reportGenerator	KRSReportGenerator	Menyimpan dependency pencetak draft KRS.
12	skemaSKS	SkemaSKS	Menyimpan aturan perhitungan SKS.
13	prasyaratValidator	PrasyaratValidator	Menyimpan dependency validasi prasyarat.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	MataKuliah(String, String, int)	Constructor	Mengisi kode, nama, dan SKS mata kuliah.
2	getJenis()	Function	Mengambil jenis mata kuliah seperti Teori, Praktikum, atau KKN.
3	alokasiAsistenLab()	Procedural	Mengalokasikan asisten lab khusus untuk praktikum.
4	cekPeralatanPraktikum()	Procedural	Mengecek peralatan praktikum.
5	hitungUKT()	Function	Menghitung UKT sesuai skema jalur mahasiswa.
6	getNamaSkema()	Function	Mengambil nama skema UKT.
7	Mahasiswa(String, String, SkemaUKT)	Constructor	Mengisi data mahasiswa dan skema UKT.
8	KRS(String, Mahasiswa)	Constructor	Membuat objek KRS untuk mahasiswa.
9	tambahMataKuliah(MataKuliah)	Procedural	Menambah mata kuliah ke daftar KRS.
10	validasi(Mahasiswa, MataKuliah, KRS)	Procedural	Mengecek data KRS sebelum mata kuliah ditambahkan.
11	hitungTotalSKS(KRS)	Function	Menghitung total SKS berdasarkan skema.
12	simpan(KRS)	Procedural	Menyimpan KRS lewat repository.
13	cetak(KRS)	Procedural	Mencetak draft KRS secara simulasi.
14	simpanKRS(KRS)	Procedural	Memanggil repository untuk simpan KRS.
15	cetakDraft(KRS)	Procedural	Memanggil report generator untuk mencetak draft.

IV. Dokumentasi dan Pembahasan Code

Bagian penjelasan file:

MataKuliah.java

```
1  public abstract class MataKuliah {
2      private String kode;
3      private String nama;
4      private int sks;
5
6      public MataKuliah(String kode, String nama, int sks) {
7          if (kode == null || kode.isBlank()) {
8              throw new IllegalArgumentException("Kode mata kuliah tidak boleh kosong");
9          }
10
11         if (nama == null || nama.isBlank()) {
12             throw new IllegalArgumentException("Nama mata kuliah tidak boleh kosong");
13         }
14
15         if (sks <= 0) {
16             throw new IllegalArgumentException("SKS harus lebih dari 0");
17         }
18
19         this.kode = kode;
20         this.nama = nama;
21         this.sks = sks;
22     }
23
24     public String getKode() {
25         return kode;
26     }
27
28     public String getNama() {
29         return nama;
30     }
31
32     public int getSks() {
33         return sks;
34     }
35
36     public abstract String getJenis();
37 }
```

Ini class induk untuk semua mata kuliah. Data yang sama seperti kode, nama, dan SKS ditaruh di sini supaya tidak ditulis ulang di Teori, Praktikum, dan KKN.

MataKuliahTeori.java

```
1 public class MataKuliahTeori extends MataKuliah {
2
3     public MataKuliahTeori(String kode, String nama, int sks) {
4         super(kode, nama, sks);
5     }
6
7     @Override
8     public String getJenis() {
9         return "Teori";
10    }
11 }
```

Class ini untuk mata kuliah teori. Isinya sederhana karena teori tidak butuh asisten lab atau cek alat praktikum.

MataKuliahPraktikum.java

```
1 public class MataKuliahPraktikum extends MataKuliah implements OperasiPraktikum {
2     public MataKuliahPraktikum(String kode, String nama, int sks) {
3         super(kode, nama, sks);
4     }
5
6     @Override
7     public String getJenis() {
8         return "Praktikum";
9     }
10
11    @Override
12    public void alokasiAsistenLab() {
13        System.out.println("Asisten lab dialokasikan untuk " + getNama());
14    }
15
16    @Override
17    public void cekPeralatanPraktikum() {
18        System.out.println("Peralatan praktikum dicek untuk " + getNama());
19    }
20 }
```

Class ini khusus praktikum. Karena praktikum memang butuh asisten lab dan peralatan praktikum, class ini implement dua interface kecil.

MataKuliahKKN.java

```
1 public class MataKuliahPraktikum extends MataKuliah implements ButuhAsistenLab, ButuhPeralatanPraktikum {
2
3     public MataKuliahPraktikum(String kode, String nama, int sks) {
4         super(kode, nama, sks);
5     }
6
7     @Override
8     public String getJenis() {
9         return "Praktikum";
10    }
11
12    @Override
13    public void alokasiAsistenLab() {
14        System.out.println("Asisten lab dialokasikan untuk " + getNama());
15    }
16
17    @Override
18    public void cekPeralatanPraktikum() {
19        System.out.println("Peralatan praktikum dicek untuk " + getNama());
20    }
21 }
```

Class ini untuk mata kuliah KKN/kuliah lapangan. KKN tetap mata kuliah, tapi tidak dipaksa punya method lab.

SkemaUKT.java

```
1 public interface SkemaUKT {
2     long hitungUKT();
3     String getNamaSkema();
4 }
```

Interface ini jadi aturan umum untuk perhitungan UKT. Jadi sistem tidak perlu if-else panjang untuk setiap jalur mahasiswa.

UKTReguler.java

```
1 public class UKTReguler implements SkemaUKT {
2     @Override
3     public long hitungUKT() {
4         return 7000000;
5     }
6
7     @Override
8     public String getNamaSkema() {
9         return "Reguler";
10    }
11 }
```

Implementasi UKT untuk mahasiswa reguler.

UKTBidikmisi.java

```
1 public class UKTBidikmisi implements SkemaUKT {
2     @Override
3     public long hitungUKT() {
4         return 0;
5     }
6
7     @Override
8     public String getNamaSkema() {
9         return "Bidikmisi";
10    }
11 }
```

Implementasi UKT untuk Bidikmisi. Di contoh ini nilainya nol.

Mahasiswa.java

```
1  public class Mahasiswa {
2      private String nim;
3      private String nama;
4      private SkemaUKT skemaUKT;
5
6      public Mahasiswa(String nim, String nama, SkemaUKT skemaUKT) {
7          this.nim = nim;
8          this.nama = nama;
9          this.skemaUKT = skemaUKT;
10     }
11
12     public String getNim() {
13         return nim;
14     }
15
16     public String getNama() {
17         return nama;
18     }
19
20     public long hitungUKT() {
21         return skemaUKT.hitungUKT();
22     }
23
24     public String getJalurMasuk() {
25         return skemaUKT.getNamaSkema();
26     }
27 }
```

Class ini menyimpan data mahasiswa dan skema UKT yang dipakai. Mahasiswa cukup memanggil `skemaUKT.hitungUKT()`.

KRS.java

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class KRS {
5      private String nomorKRS;
6      private Mahasiswa mahasiswa;
7      private ArrayList<MataKuliah> daftarMataKuliah;
8
9      public KRS(String nomorKRS, Mahasiswa mahasiswa) {
10         this.nomorKRS = nomorKRS;
11         this.mahasiswa = mahasiswa;
12         this.daftarMataKuliah = new ArrayList<>();
13     }
14
15     public String getNomorKRS() {
16         return nomorKRS;
17     }
18
19     public Mahasiswa getMahasiswa() {
20         return mahasiswa;
21     }
22
23     public void tambahMataKuliah(MataKuliah mataKuliah) {
24         daftarMataKuliah.add(mataKuliah);
25     }
26
27     public List<MataKuliah> getDaftarMataKuliah() {
28         return daftarMataKuliah;
29     }
30 }
```

Class ini menyimpan data KRS: ID KRS, mahasiswa, dan daftar mata kuliah. Daftar mata kuliah dibuat dinamis dengan ArrayList.

SkemaSKS.java

```
1 public interface SkemaSKS {  
2     int hitungTotalSKS(KRS krs);  
3 }
```

Interface untuk menghitung total SKS. Ini disiapkan karena soal menyebut akan ada skema MBKM.

SkemaSKSNormal.java

```
1 public class SkemaSKSNormal implements SkemaSKS {  
2     @Override  
3     public int hitungTotalSKS(KRS krs) {  
4         int total = 0;  
5  
6         for (MataKuliah mk : krs.getDaftarMataKuliah()) {  
7             total += mk.getSks();  
8         }  
9  
10        return total;  
11    }  
12 }
```

Skema normal menjumlahkan semua SKS mata kuliah apa adanya.

SkemaSKSMBKM.java

```
1 public class SkemaSKSMBKM implements SkemaSKS {
2     @Override
3     public int hitungTotalSKS(KRS krs) {
4         int total = 0;
5
6         for (MataKuliah mk : krs.getDaftarMataKuliah()) {
7             if (mk instanceof MataKuliahKKN) {
8                 total += 20;
9             } else {
10                 total += mk.getSks();
11             }
12         }
13
14         return total;
15     }
16 }
```

Contoh skema MBKM. Di contoh ini KKN dikonversi menjadi 20 SKS, hanya untuk menunjukkan bahwa aturan SKS bisa diganti tanpa mengubah KRSService.

KRSRepository.java

```
1 public interface KRSRepository {
2     void simpan(KRS krs);
3 }
```

Interface untuk penyimpanan KRS. Tujuannya supaya service tidak terikat langsung ke MySQL.

MySQLKRSRepository.java

```
1 public class MySQLKRSRepository implements KRSRepository {
2     @Override
3     public void simpan(KRS krs) {
4         System.out.println("KRS " + krs.getNomorKRS() + " disimpan ke MySQL.");
5     }
6 }
```

Implementasi repository untuk simulasi penyimpanan ke MySQL.

CloudNoSQLKRSRepository.java

```
1 public class CloudNoSQLKRSRepository implements KRSRepository {
2     @Override
3     public void simpan(KRS krs) {
4         System.out.println("KRS " + krs.getNomorKRS() + " disimpan ke Cloud NoSQL.");
5     }
6 }
```

Implementasi repository untuk simulasi penyimpanan ke Cloud NoSQL.

KRSReportGenerator.java

```
1 public interface KRSReportGenerator {
2     void cetak(KRS krs);
3 }
```

Interface untuk pembuatan laporan atau draft KRS.

PDFKRSReportGenerator.java



```
1 public class PDFKRSReportGenerator implements KRSReportGenerator {
2     @Override
3     public void cetak(KRS krs) {
4         System.out.println("Draft PDF KRS untuk " + krs.getMahasiswa().getNama());
5
6         for (MataKuliah mk : krs.getDaftarMataKuliah()) {
7             System.out.println("- " + mk.getKode() + " | " + mk.getNama() + " | " + mk.getSks() + " SKS");
8         }
9     }
10 }
```

Implementasi generator laporan PDF secara simulasi lewat output console. Tidak benar-benar membuat file PDF.

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Mahasiswa mahasiswa = new Mahasiswa("105224041", "Arif Mufti Tharsa", new UKTReguler());
4
5         MataKuliah mk1 = new MataKuliahTeori("IF101", "Rekayasa Perangkat Lunak", 3);
6         MataKuliah mk2 = new MataKuliahPraktikum("IF102", "Praktikum PBO", 1);
7         MataKuliah mk3 = new MataKuliahKKN("UP301", "Kuliah Kerja Nyata", 4);
8
9         KRS krs = new KRS("KRS001", mahasiswa);
10
11         KRSRepository repository = new CloudNoSQLKRSRepository();
12         KRSReportGenerator report = new PDFKRSReportGenerator();
13         SkemaSKS skemaNormal = new SkemaSKSNormal();
14
15         KRSService service = new KRSService(repository, report, skemaNormal);
16
17         service.tambahMataKuliah(krs, mk1);
18         service.tambahMataKuliah(krs, mk2);
19         service.tambahMataKuliah(krs, mk3);
20
21         System.out.println("--- Data Krs ---");
22         System.out.println("Nama Mahasiswa: " + mahasiswa.getNama());
23         System.out.println("NIM: " + mahasiswa.getNim());
24         System.out.println("Jalur UKT: " + mahasiswa.getJalurMasuk());
25         System.out.println("Tagihan UKT: " + service.hitungTagihanUKT(mahasiswa));
26         System.out.println("Total SKS: " + service.hitungTotalSKS(krs));
27         System.out.println();
28
29         System.out.println("---- Daftar Mata Kuliah ---");
30         for (MataKuliah mk : krs.getDaftarMataKuliah()) {
31             System.out.println(mk.getKode() + " - " + mk.getNama() + " - " + mk.getJenis());
32
33             if (mk instanceof OperasiPraktikum) {
34                 OperasiPraktikum praktikum = (OperasiPraktikum) mk;
35                 praktikum.alokasiAsistenLab();
36                 praktikum.cekPeralatanPraktikum();
37             }
38         }
39
40         System.out.println();
41         service.simpanKRS(krs);
42         service.cetakDraftKRS(krs);
43
44         System.out.println();
45         System.out.println("---- Pindah Database ---");
46         KRSService serviceMySQL = new KRSService(new MySQLKRSRepository(), report, skemaNormal);
47         serviceMySQL.simpanKRS(krs);
48
49         System.out.println();
50         System.out.println("--- Skema Mbkm ----");
51         KRSService serviceMBKM = new KRSService(repository, report, new SkemaSKSMBKM());
52         System.out.println("Total SKS MBKM: " + serviceMBKM.hitungTotalSKS(krs));
53
54         System.out.println();
55         System.out.println("--- UKT Bidikmisi ---");
56         Mahasiswa mahasiswa2 = new Mahasiswa("105224099", "Budi", new UKTBidikmisi());
57         System.out.println("Nama Mahasiswa: " + mahasiswa2.getNama());
58         System.out.println("Jalur UKT: " + mahasiswa2.getJalurMasuk());
59         System.out.println("Tagihan UKT: Rp" + mahasiswa2.hitungUKT());
60     }
61 }
```

File ini cuma dipakai buat demo pemakaian desain baru. Di sini dibuat mahasiswa, mata kuliah, KRS, repository, report generator, skema SKS, dan service. Jadi Main bukan tempat logika utama.

V. Pembahasan SOLID

Prinsip	Penerapan	Catatan
SRP	SistemKRSManager lama dipecah. Validasi, UKT, repository, report, dan service dibuat terpisah.	Sesuai dengan masalah yang dijelaskan pada soal.
OCP	Jika ada jalur UKT baru, cukup buat class baru yang implements SkemaUKT. Jika ada skema SKS baru, cukup buat class baru yang implements SkemaSKS.	Tidak perlu bongkar class lama setiap ada aturan baru.
LSP	MataKuliahTeori, MataKuliahPraktikum, dan MataKuliahKKN tetap bisa dipakai sebagai MataKuliah tanpa memaksa method yang tidak cocok.	Sesuai dengan masalah yang dijelaskan pada soal.
ISP	Interface praktikum dipisah menjadi ButuhAsistenLab dan ButuhPeralatanPraktikum, jadi Teori dan KKN tidak dipaksa implement method lab.	Ini menjawab masalah interface OperasiMataKuliah yang terlalu besar.
DIP	KRSService bergantung pada interface seperti KRSRepository, KRSReportGenerator, SkemaSKS, dan PrasyaratValidator, bukan class konkret seperti MySQL langsung.	Ini yang membuat migrasi MySQL ke Cloud NoSQL jadi lebih gampang.

VI. Kesimpulan

Dari tugas ini, masalah utama sistem lama adalah terlalu banyak tanggung jawab yang ditaruh di SistemKRSManager. Akibatnya, perubahan kecil seperti ubah format PDF bisa merusak bagian database. Selain itu, interface mata kuliah terlalu besar, perhitungan UKT terlalu banyak if-else, dan service bergantung langsung pada MySQL.

Rancangan baru memecah kode menjadi beberapa bagian: mata kuliah, UKT, KRS, validasi, repository, report, dan service. Dengan desain ini, migrasi dari MySQL ke Cloud NoSQL cukup dilakukan dengan mengganti implementasi KRSRepository. Penambahan jalur UKT atau skema SKS MBKM juga bisa dilakukan dengan menambah class baru, bukan membongkar class lama. Jadi, desain ini lebih rapi, lebih mudah dibaca, dan lebih aman untuk dikembangkan dibanding sistem lama yang semua logic-nya digabung di satu class besar.